SRI RAMACHANDRA
INSTITUTE OF HIGHER EDUCATION AND RESEARCH
(Category - I Deemed to be University) Porur, Chennai
SRI RAMACHANDRA FACULTY OF ENGINEERING AND TECHNOLOGY

RETAIL MARKET-BASKET ANALYSIS

ADA23CL302 – DATA MINING LABORATORY
PROJECT REPORT

Submitted by

R Bavadhaarini – E0324001

M Lishalini – E0324002

In partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

(Artificial Intelligence and Data Analytics)

Sri Ramachandra Faculty of Engineering and Technology
Sri Ramachandra Institute of Higher Education and Research,
Porur, Chennai - 600116

October 2026

SRI RAMACHANDRA
INSTITUTE OF HIGHER EDUCATION AND RESEARCH
(Category - I Deemed to be University) Porur, Chennai
SRI RAMACHANDRA FACULTY OF ENGINEERING AND TECHNOLOGY

BONAFIDE CERTIFICATE

We, **R Bavadhaarini (E0324001) & M Lishalini (E0324002)** hereby declare that the Project Report on **"Retail Market-Basket Analysis"** is submitted by us under the guidance of Prof. P. Deepika for ADA23CL302 – Data Mining Laboratory, at Sri Ramachandra Faculty of Engineering and Technology, in partial fulfilment of Semester 5, Third Year B.Tech Degree course in the Department of Computer Science and Engineering during the academic year 2026-2027.

Signature of the Supervisor

Prof. P. Deepika
Assistant Professor,
Department of Artificial Intelligence and Data Analytics,
Sri Ramachandra Faculty of Engineering and Technology,
SRIHER, Porur, Chennai - 600116.

Evaluation Date:

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

We take this opportunity to express our gratitude to Prof. T. Ragunathan, Dean, and Prof. A. Saravanan, Vice Dean, Sri Ramachandra Faculty of Engineering and Technology, SRIHER, for providing all the facilities to complete this project work successfully.

We express our sincere gratitude to Dr. A. Sathya, Programme Coordinator, Department of Artificial Intelligence and Data Analytics, SRET, for providing the required support and facilities for carrying out this study.

We also extend our sincere gratitude to Prof. P. Deepika, Course Coordinator, whose guidance and continuous support were vital to the completion of this project.

We are grateful to all the members of the Sri Ramachandra Faculty of Engineering and Technology, our beloved parents and our friends for their support, which helped us to overcome obstacles during the study.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	**1**
1	**INTRODUCTION**	**2**
1.1	Problem Statement	2
1.2	Motivation	2
1.3	Objectives	3
1.4	Scope of the Project	3
2	**DATASET**	**4**
2.1	Dataset Source	4
2.2	Number of Records	4
2.3	Attributes	5
2.4	Analytical Targets	5
2.5	Statistical Summary	6
3	**DATA PREPROCESSING**	**7**
3.1	Missing Values	7
3.2	Cleaning	7
3.3	Transformation and Data Reduction	8
3.4	Normalization and Partitioning	8
4	**DATA MINING TECHNIQUES**	**10**
4.1	Association Rule Mining Concepts	10
4.2	Apriori Frequent Itemset Mining	10
4.3	Rule Generation and Interestingness Measures	11
4.4	Next-Best-Offer Recommendation	12
4.5	Threshold Selection and Complexity	13
5	**IMPLEMENTATION**	**14**
5.1	Introduction	14
5.2	Development Environment and Tools	14
5.3	Dataset Loading and Inspection	14
5.4	Data Preprocessing	15
5.5	Engine Structure and Functions	15
5.6	Implementation of the Mining Modules	15
5.7	Front-End Dashboard	16
5.8	Export of Results	17
5.9	Verification and Testing	17
5.10	Implementation Pipeline	17
6	**RESULTS**	**19**
6.1	Introduction	19
6.2	Data Analysis	19
6.3	Results on the Reference Dataset	20
6.4	Frequent Itemsets and Apriori Pruning	21
6.5	Association Rules on the Supermarket Dataset	22
6.6	Association Rules on the UCI Online Retail Dataset	24

6.7 Recommendation Results 25
6.8 Sensitivity to the Thresholds 26
6.9 Performance and Verification 27
6.10 Effect of Live Transactions 27
6.11 Summary of Results 28
7 COMPARISON AND DISCUSSION 29
7.1 Comparison of the Datasets 29
7.2 Choosing the Thresholds 29
7.3 Apriori Compared with Alternative Algorithms 30
7.4 Advantages 30
7.5 Limitations 31
8 CONCLUSION AND FUTURE SCOPE 32
8.1 Conclusion 32
8.2 Future Scope 32
REFERENCES 34
APPENDIX: KEY SOURCE CODE 35
A.1 Universal transaction loader (load_transactions) 35
A.2 Live POS write (append_transaction) 35
A.3 Apriori frequent-itemset mining (apriori) 36
A.4 Association rule generation (generate_rules) 37
A.5 Next-best-offer recommender (recommend) 38
A.6 Cached mining call in the dashboard (app.py) 39

LIST OF FIGURES

Figure No.	Title of the Figure	Page No.
Figure 2.1	Distribution of Basket Sizes	6
Figure 5.1	System Architecture of the Retail Market-Basket System	18
Figure 6.1	Most Frequent Products in the Supermarket Dataset	20
Figure 6.2	Candidate Itemsets Generated and Evaluated at Each Level	22
Figure 6.3	Support versus Confidence of the Supermarket Rules	23
Figure 6.4	Ten Strongest Supermarket Rules by Lift	23
Figure 6.5	Ten Strongest UCI Rules by Lift	25
Figure 6.6	Effect of Minimum Support on Output Size and Run Time	26

ABSTRACT

Every sale in a shop is recorded as a basket of products, and the resulting transaction logs grow to thousands or millions of baskets. Reading them by eye reveals nothing about which products customers tend to buy together, yet that knowledge drives shelf layout, promotions, bundling and cross-selling. This project applies association rule mining, a core data mining technique, to retail transaction data and delivers it as a working analytics system, the Retail Market-Basket Analysis.

The central component is an Apriori frequent-itemset engine written from scratch in Python, without mlxtend, apyori or any other mining library. It uses level-wise candidate generation with downward-closure pruning, records how many candidates are generated, pruned and evaluated at each level, and derives association rules whose antecedent and consequent supports are read from a hash map in constant time instead of re-scanning the transactions. Every rule carries five interestingness measures: support, confidence, lift, leverage and conviction. A universal loader accepts the common Kaggle and UCI transaction layouts as well as headerless basket-per-row files, and a Streamlit dashboard provides a live point-of-sale checkout with next-best-offer recommendations, a searchable rules explorer, Plotly visual analytics and a view of the pruning telemetry.

The system was evaluated on three datasets: a 15-basket reference dataset, the public 9,835-basket supermarket Groceries dataset, and 3,000 invoices from the UCI Online Retail dataset. On the supermarket data, with minimum support 2%, minimum confidence 30% and minimum lift 1.2, the engine found 122 frequent itemsets and 37 association rules in about 0.68 seconds. Whole Milk is the consequent of 24 of the 37 rules, and the strongest rule, Other Vegetables and Whole Milk implying Root Vegetables, has a lift of 2.84. Apriori pruning removed 409 of the 477 candidate three-itemsets (85.7%) before any transaction scan. On the UCI data (minimum support 2%, confidence 40%, lift 1.5) the engine found 516 frequent itemsets and 132 rules with lifts from 2.7 to 27.4, dominated by colour and size variants of the same product. The engine was verified against exhaustive enumeration on the reference dataset and against direct recounts on the supermarket data, and all checks agreed exactly. All figures in this report come from running the submitted code on the included data.

Keywords: Market Basket Analysis, Association Rule Mining, Apriori Algorithm, Frequent Itemsets, Support, Confidence, Lift, Recommender System, Streamlit.

CHAPTER 1
INTRODUCTION

1.1 Problem Statement

A retail business generates a continuous stream of point-of-sale records. Each purchase is a small set of products bought together, and the complete history contains valuable information about customer behaviour: which items are bought together, which purchases predict other purchases, and how strong these links are compared with what chance alone would produce. An analyst looking at raw transaction rows cannot see this structure. With 169 product categories there are already 14,196 possible pairs and more than 790,000 possible triples, and with the 2,955 distinct products in the UCI Online Retail sample there are more than four million possible pairs, so exhaustive inspection is impossible.

The problem addressed in this project is to build a system that reads transaction data in common CSV layouts, mines frequent itemsets and association rules efficiently, measures each rule with several standard interestingness measures so that useful rules can be told apart from coincidences, turns the rules into real-time product recommendations for a customer's current basket, and presents everything through an interactive dashboard in which the thresholds can be changed and the effect seen immediately.

1.2 Motivation

Market-basket analysis is one of the best-known applications of data mining. The result that a customer who buys product A is much more likely than average to buy product B directly supports decisions such as placing the two products near each other, bundling them, offering a coupon on B to customers who put A in their cart, or planning stock levels. Because the technique finds patterns without predefined labels, it can reveal combinations that nobody thought to look for.

There are two further motivations. The first is academic: Apriori is the textbook algorithm for frequent-itemset mining, and implementing it from scratch, including the pruning step and the rule-generation step, shows how the algorithm works instead of hiding it inside a library call. The project therefore keeps the mining engine free of external mining packages and instruments it so that the saving produced by the Apriori property can be observed and counted. The second is honesty and reproducibility: the datasets used are public benchmark datasets that can be downloaded again by the included script, and every number reported is produced by running the submitted code.

1.3 Objectives

The main objectives of the project are:

- To implement the Apriori algorithm from scratch, with level-wise candidate generation, downward-closure pruning and support counting on frozenset baskets.
- To record pruning telemetry (candidates generated, pruned, evaluated and frequent itemsets at each level) so that the efficiency of the Apriori property can be measured.
- To generate association rules with five interestingness measures (support, confidence, lift, leverage and conviction), using constant-time hash-map lookups for the antecedent and consequent supports.
- To load transaction data from several real-world CSV layouts (TransactionID and Item, InvoiceNo and Description, Member_number and itemDescription) and from headerless basket-per-row files.
- To evaluate the system on real public datasets: the 9,835-basket supermarket Groceries dataset and invoices from the UCI Online Retail dataset, together with a small reference dataset for baseline checks.
- To provide a real-time next-best-offer recommender that suggests products for the items currently in a customer cart.
- To build an interactive dashboard with live checkout, rule exploration with CSV export, visual analytics and algorithm telemetry.
- To verify the correctness of the engine against independent exhaustive and direct-count calculations.

1.4 Scope of the Project

The project covers association rule mining on transaction data in which each basket is treated as a set of distinct products. Quantities, prices, profit margins, customer attributes and time of purchase are not used, so sequential patterns, seasonal effects and revenue-weighted rules are outside the present scope and are listed as future work. Supervised prediction of individual customer behaviour is also out of scope. The datasets are public benchmark datasets, so the findings describe what the algorithm discovers in those data and should not be read as the buying behaviour of any particular store or country.

CHAPTER 2
DATASET

2.1 Dataset Source

The project uses three datasets, all kept in the `data/` folder. Two are real public benchmark datasets that the script `fetch_public_data.py` downloads and converts into a common two-column format; the third is the small reference dataset used for baseline checks.

- **Supermarket POS (Groceries) dataset.** The widely used Groceries data: one month of point-of-sale transactions from a grocery outlet, published as a one-basket-per-line file in the public repository that accompanies the book *Machine Learning with R*. The script downloads it and writes one row per basket-item pair with identifiers `POS_0001`, `POS_0002` and so on. The items are product categories (for example Whole Milk, Other Vegetables, Yogurt), not individual brands.
- **UCI Online Retail dataset.** Transactions of a UK-based online gift retailer from the UCI Machine Learning Repository, obtained from a public mirror. The script keeps the first 3,000 non-cancelled invoices, skips cancelled invoices (invoice numbers starting with C) and non-product lines (Postage, Dotcom Postage, Manual, Discount), and writes the invoice number and the title-cased product description.
- **Reference dataset.** A hand-made set of 15 baskets over six everyday products (Bread, Milk, Butter, Eggs, Coffee, Sugar). It is small enough to check every result by hand and contains one strong pair (Coffee and Sugar) on purpose.

The loader is not limited to these files. Any CSV with a recognisable transaction-identifier column and item column, or any headerless file with one basket per row, can be uploaded in the dashboard and is mined in exactly the same way (Table 2.3).

Note on live baskets. The dashboard can append live checkout and simulated baskets to the active CSV file (Section 5.6.4). The copies of `real_groceries_9835.csv` and `transactions.csv` in the submitted archive contain such test additions, identified by the prefix `LIVE_` (28 baskets and 1 basket respectively). All results in this report were computed on the original baskets only, with the `LIVE_` rows removed, so that they match the published dataset sizes. Section 6.10 shows the effect of the additions.

2.2 Number of Records

Table 2.1: Datasets used in the project

Dataset	Baskets	Distinct items	Item records	Notes
Reference (`transactions.csv`)	15	6	38	Toy data for baseline checks; 1 to 4 items per basket
Supermarket POS (`real_groceries_9835.csv`)	9,835	169	43,367	Real grocery transactions; items are product categories
UCI Online Retail (`uci_online_retail.csv`)	3,000	2,955	83,547 (81,880 after de-duplication)	First 3,000 non-cancelled invoices; many wholesale-sized baskets
User-uploaded CSV	Variable	Variable	Variable	Any supported layout; saved as `uploaded_custom.csv`

Each basket represents one customer purchase (one till receipt or one invoice). Item records are the rows of the CSV file, one for each product in a basket.

2.3 Attributes

After conversion every dataset has the same logical schema of two attributes. The relationship is one-to-many: one transaction identifier appears on as many rows as the basket has products.

Table 2.2: Attributes of the transaction dataset

Attribute	Type	Description
TransactionID	Categorical	Identifier of the basket: POS number, invoice number or `T`-number. Rows with the same identifier belong to one basket.
Item	Categorical	Name of one product in the basket. Names are stripped and converted to title case so that spelling variants are merged.

Because real files use different column names, the loader checks a list of accepted headers (Table 2.3). If no pair of recognised columns is found, the file is read as a headerless basket-per-row file, in which every row is one basket and every non-empty cell is an item.

Table 2.3: Column names recognised by the universal loader

Source layout	Transaction column	Item column
Standard (this project)	TransactionID	Item
UCI Online Retail	InvoiceNo	Description
Kaggle Bread Basket	Transaction	Item
Kaggle Groceries	Member_number	itemDescription
Other accepted names	Order_ID, BillNo	Product, Itemname
Headerless basket-per-row file	Row number (T0001, T0002, ...)	Each non-empty cell in the row

2.4 Analytical Targets

Association rule mining is unsupervised, so there is no labelled target variable to predict. The patterns to be discovered are the frequent itemsets (sets of products that appear together in at least a minimum share of baskets) and the association rules of the form X implies Y derived from them. Each rule is described by five numbers: support, confidence, lift, leverage and conviction, defined in Section 4.3. A rule is kept only if its confidence and lift reach the thresholds chosen by the user. The recommender then turns the rules into a ranked list of products for a given cart.

2.5 Statistical Summary

Table 2.4 summarises the basket sizes. The supermarket baskets are small (mean 4.4 items, median 3) and 22.0% contain a single item, whereas the online-retail invoices are much larger (mean 27.3 items, median 14, maximum 672) because many customers are trade buyers purchasing in bulk. This difference matters for the mining: large baskets create many co-occurring pairs, so the UCI data produces far more candidate pairs than the supermarket data at the same support threshold (Section 6.4).

Table 2.4: Basket-size statistics (items per basket)

Dataset	Baskets	Mean	Median	Min	Max	Single-item baskets
Reference	15	2.53	2	2	4	0 (0.0%)
Supermarket POS	9,835	4.41	3	1	32	2,159 (22.0%)
UCI Online Retail	3,000	27.29	14	1	672	303 (10.1%)

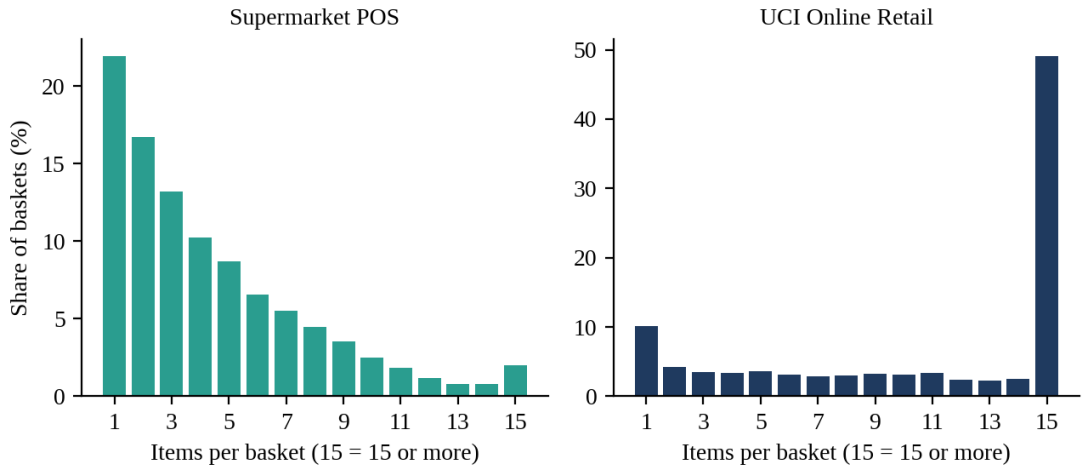


Figure 2.1: Distribution of Basket Sizes

CHAPTER 3
DATA PREPROCESSING

Transaction files from different sources differ in layout, spelling and the kind of rows they contain. The data here is categorical and has no numeric measurements, so preprocessing does not involve scaling or imputing numbers. Instead, it has to convert rows into baskets, remove rows that are not real purchases and make item names consistent. This is done in two places: once when the benchmark datasets are downloaded (`fetch_public_data.py`) and again every time a file is read by the loader (`load_transactions` in `main.py`).

3.1 Missing Values

Transaction data rarely has missing numeric values, but it does contain rows that carry no usable item information. The loader treats the following as missing and skips them: an empty transaction identifier, an empty item name, and item names equal to None, NaN or Adjustment (case-insensitive), which are placeholders used in some Kaggle and UCI files. Because a basket is simply the set of its items, a missing item does not need to be imputed; the remaining items of the basket still describe the purchase. Baskets that end up with no items are dropped. In the headerless basket-per-row layout, empty cells (which appear when baskets have different lengths) are ignored in the same way.

3.2 Cleaning

The cleaning applied by the loader, in order, is as follows.

1. **Reading the file.** The file is opened with UTF-8 (with or without a byte-order mark) and undecodable characters are ignored, so files exported from spreadsheet programs load without errors.
2. **Layout detection.** The header row is searched for a known transaction column and item column (Table 2.3). If none is found, the file is read as a headerless basket-per-row file.
3. **Removal of cancelled invoices.** Transaction identifiers starting with the letter C are cancellations in the UCI data and are skipped. The download script also removes them, together with non-product lines (Postage, Dotcom Postage, Manual, Discount).
4. **Removal of placeholder items.** Items equal to None, NaN or Adjustment are skipped.
5. **Name normalisation.** Item names are stripped of surrounding spaces and converted to title case, so that for example whole milk and Whole Milk count as one product.

6. **De-duplication within a basket.** Items are collected into a set per basket, so a product appearing on several lines of the same invoice is counted once. Association rule mining asks whether a product is present in a basket, not how many units were bought.
7. **Canonical ordering.** The items of each basket are sorted so that itemsets can be compared and used as dictionary keys.

Table 3.1: Effect of loading and cleaning on each dataset

Dataset	Rows in CSV	Baskets	Basket-item pairs after cleaning	Removed as duplicates
Reference	38	15	38	0
Supermarket POS	43,367	9,835	43,367	0
UCI Online Retail	83,547	3,000	81,880	1,667

The supermarket and reference files contain no repeated items inside a basket, so nothing is removed. In the UCI file 1,667 rows repeat a product already present on the same invoice (the same description entered on more than one line) and are merged. The download script had already removed cancelled invoices and non-product lines before the file was saved.

3.3 Transformation and Data Reduction

After cleaning, the data is transformed into the form the mining algorithm needs:

- **From long format to baskets.** The two-column file (one row per item) is grouped by transaction identifier into a list of baskets, each a sorted list of items (the horizontal transaction format used by Apriori).
- **Frozenset baskets.** Inside the Apriori function every basket is converted to a frozenset, which makes the test "is this candidate itemset contained in this basket?" a fast subset operation.
- **Itemsets as sorted tuples.** Itemsets are stored as sorted tuples, so the same set of products always has the same key in the dictionary of frequent itemsets and its support can be looked up directly.
- **Support as a fraction.** Support is stored as a fraction of the total number of baskets, so the minimum support threshold is independent of the dataset size.
- **Data reduction.** The UCI source file is limited to its first 3,000 valid invoices to keep the pure-Python mining time practical. In addition, the minimum support threshold itself acts as a reduction step: items below it are never extended into larger itemsets.

3.4 Normalization and Partitioning

Normalisation. The data contains no numeric attributes, so no scaling of values is required. The relevant normalisation is the standardisation of item names described in Section 3.2 (stripping and title case), which prevents one product from being split into several differently spelt items.

Partitioning. Association rule mining is descriptive and has no labelled target, so a training and test split is not applied: every algorithm run uses all baskets of the selected dataset, and there is no train-to-test leakage to guard against. Rule quality is judged with the interestingness measures themselves (support, confidence, lift, leverage and conviction) and the engine is judged by verification against independent calculations (Section 6.9). The only partitioning in the project is logical: the three datasets are mined separately with thresholds suited to their size, and the added LIVE baskets can be identified and excluded by their prefix.

CHAPTER 4
DATA MINING TECHNIQUES

The project combines three connected techniques. Apriori finds the sets of products that are bought together often enough to matter. Rule generation turns those itemsets into if-then rules and scores each rule with five measures. The recommender then applies the rules to a customer's current cart to suggest the next product.

4.1 Association Rule Mining Concepts

Let I be the set of all items sold and T the set of all transactions (baskets), where each transaction t is a subset of I. An **itemset** is any set of items. An **association rule** is an implication $X \rightarrow Y$, where X and Y are disjoint itemsets, X is called the antecedent (the IF part) and Y the consequent (the THEN part). For example, the rule Yogurt → Whole Milk says that baskets containing yogurt tend also to contain whole milk. The rule is a statement about co-occurrence in past baskets and does not claim that buying yogurt causes the purchase of milk.

Mining is done in two stages. First, all itemsets whose support reaches a user-chosen minimum are found (frequent itemsets). Second, rules are formed from the frequent itemsets and only those meeting the minimum confidence and minimum lift are kept. Frequent-itemset mining is the expensive stage, because the number of possible itemsets grows exponentially with the number of items. Apriori makes it practical by exploiting one property.

4.2 Apriori Frequent Itemset Mining

The **Apriori property** (downward closure) states that every non-empty subset of a frequent itemset must itself be frequent. Equivalently, if an itemset is infrequent, every larger itemset that contains it must also be infrequent and need not be examined. Apriori uses this to search level by level, where level k holds the itemsets with k items.

The algorithm implemented in `apriori()` works as follows:

1. **Level 1.** Count how often each item occurs in one pass over the baskets. Items whose support reaches the minimum form the frequent 1-itemsets L1.
2. **Candidate generation (join).** From the frequent (k−1)-itemsets, build candidate k-itemsets by merging pairs of them and keeping the unions that have exactly k items.

3. **Pruning.** Discard every candidate that has a (k−1)-subset which is not frequent. These candidates cannot be frequent by the Apriori property, so they are removed without counting.
4. **Support counting.** For each surviving candidate, count the baskets that contain it (a subset test on frozenset baskets) and keep the candidates whose support reaches the minimum. These form Lk.
5. **Repeat** steps 2 to 4 until no candidates remain. The union of all Lk is the set of frequent itemsets, stored in a dictionary that maps each itemset to its support.

At level 2 every pair of frequent items is a candidate, because all single-item subsets are frequent by construction. If L1 has n items, the number of level-2 candidates is therefore n(n−1)/2, which gives 1,711 candidates for the 59 frequent supermarket items and 69,751 candidates for the 374 frequent UCI items. From level 3 onwards the pruning step becomes effective, and Section 6.4 reports how many candidates it removes. For every level the function records the number of candidates generated, the number remaining after pruning, the number pruned and the number of frequent itemsets found. This **telemetry** makes the saving visible and is shown in the dashboard.

4.3 Rule Generation and Interestingness Measures

For every frequent itemset with at least two items, the function `generate_rules()` forms every rule X → Y in which X is a non-empty proper subset of the itemset and Y is the rest of the itemset. Because all subsets of a frequent itemset are frequent (Apriori property), the supports of X and Y are already stored in the dictionary of frequent itemsets and are fetched in constant time; the baskets are not scanned again. Five measures are calculated for each rule.

Support is the share of all baskets that contain both X and Y. It shows how common the combination is.

$$\text{Support}(X \to Y) = \frac{|\{\, t \in T : X \cup Y \subseteq t \,\}|}{|T|}$$

Confidence is the conditional probability of Y given X: the share of baskets containing X that also contain Y.

$$\text{Confidence}(X \to Y) = \frac{\text{Support}(X \cup Y)}{\text{Support}(X)}$$

Lift compares the confidence with the overall popularity of Y. A value of 1 means X and Y occur together exactly as often as independence would predict; values above 1 indicate a positive association.

$$\text{Lift}(X \to Y) = \frac{\text{Confidence}(X \to Y)}{\text{Support}(Y)}$$

Leverage is the difference between the observed co-occurrence and the co-occurrence expected under independence, expressed as a share of all baskets.

$$\text{Leverage}(X \to Y) = \text{Support}(X \cup Y) - \text{Support}(X) \times \text{Support}(Y)$$

Conviction measures how much more often the rule would be wrong if X and Y were independent. It is infinite for a rule that is never wrong, and the engine caps it at 99.99 when the confidence is 1.

$$\text{Conviction}(X \to Y) = \frac{1 - \text{Support}(Y)}{1 - \text{Confidence}(X \to Y)}$$

Table 4.1: Interpretation of the five measures

Measure	Range	Interpretation
Support	0 to 1	Higher means the combination is common and the rule is based on many baskets.
Confidence	0 to 1	Higher means Y is more likely when X is present. It is inflated when Y is a very popular product.
Lift	0 to infinity	Above 1: positive association. Equal to 1: independence. Below 1: negative association.
Leverage	-0.25 to 0.25	Zero means independence; positive values show co-occurrence above chance. Its size depends on how common the items are.
Conviction	0 to infinity	Above 1 means the rule is more reliable than chance; larger is stronger.

Worked example. On the supermarket data there are N = 9,835 baskets; 1,372 contain Yogurt, 2,513 contain Whole Milk and 551 contain both. For the rule Yogurt → Whole Milk: support = 551 / 9,835 = 0.0560 (5.60%); confidence = 551 / 1,372 = 0.4016 (40.2%); Support(Whole Milk) = 2,513 / 9,835 = 0.2555, so lift = 0.4016 / 0.2555 = 1.57; leverage = 0.0560 − 0.1395 × 0.2555 = 0.0204; and conviction = (1 − 0.2555) / (1 − 0.4016) = 1.24. A yogurt buyer is 1.57 times as likely to buy whole milk as a typical customer. These are the values the engine reports for this rule.

Rules that meet both thresholds are returned sorted by lift and then by confidence, so the strongest associations appear first.

4.4 Next-Best-Offer Recommendation

The recommender uses the mined rules directly. Given the items in the cart, it selects every rule whose antecedent is entirely contained in the cart, takes the consequent items that are not already in the cart, and for each such product keeps the rule with the highest confidence (ties broken by lift). The products are then ranked by confidence and lift, and each recommendation shows the rule that triggered it. Rules with several antecedent items, such as Root Vegetables and Whole Milk implying Other Vegetables, act only when all their antecedent items are in the cart and are therefore more specific.

4.5 Threshold Selection and Complexity

The three thresholds control the output. Minimum support decides which itemsets are frequent and has by far the largest effect on run time: lowering it admits more frequent items, which creates quadratically more candidate pairs. Minimum confidence removes weak rules, and minimum lift removes rules that merely reflect popular products. Each dataset has its own defaults (Table 4.2), chosen because a 15-basket dataset needs a high support (20%) to produce anything meaningful, whereas the large datasets contain many rare items and need a low support (2%) to find enough patterns without exploding the number of candidates. The dashboard sliders cover support 1% to 60%, confidence 10% to 100% and lift 0.5 to 5.0.

Table 4.2: Default thresholds used for each dataset

Dataset	Minimum support	Minimum confidence	Minimum lift
Reference (also the CLI default)	20%	50%	1.00
Supermarket POS	2%	30%	1.20
UCI Online Retail	2%	40%	1.50
Uploaded CSV	2%	30%	1.10

The cost of Apriori is dominated by support counting. Each candidate must be tested against every basket, so the number of subset tests is the number of candidates times the number of baskets. At level 2 this is $1{,}711 \times 9{,}835 \approx 16.8$ million tests for the supermarket data and $69{,}751 \times 3{,}000 \approx 209$ million tests for the UCI data. The measured run times in Section 6.9 follow this ratio closely.

CHAPTER 5
IMPLEMENTATION

5.1 Introduction

The system is implemented in Python as a two-part application: a mining engine (`main.py`) with no web or visualisation dependencies, and a Streamlit dashboard (`app.py`) that imports the engine. A third script (`fetch_public_data.py`) downloads and formats the benchmark datasets. Keeping the engine independent of the dashboard means that `python main.py` runs in any terminal, the algorithms can be explained and tested on their own, and the same functions serve both the command-line and the web interface.

5.2 Development Environment and Tools

Table 5.1: Technologies used in the project

Layer	Technology	Purpose
Language	Python 3.10+	Mining engine, data download script and dashboard.
Mining engine	Python standard library (`csv`, `itertools`, `collections`, `time`, `pathlib`)	File loading, combinations, counting, timing. No external mining library is used.
Data handling	pandas 2.2+	Tables shown in the dashboard and CSV export of rules.
Dashboard	Streamlit 1.36+	Single-page web interface with sidebar, tabs, sliders, cache and file upload.
Visualisation	Plotly 5.19+ (Plotly Express)	Bar chart of product frequencies and Support vs Confidence bubble chart.
Data acquisition	`urllib`, `csv`	Download and reformatting of the public benchmark datasets.
Documentation	README.md, AGENTS.md	Usage manual and the rules that protect the from-scratch design of the engine.

5.3 Dataset Loading and Inspection

In the dashboard the user chooses one of four sources in the sidebar: the supermarket dataset, the UCI dataset, an uploaded CSV file or the reference dataset. If a benchmark file is missing, the dashboard calls the matching download function of `fetch_public_data.py`. An uploaded file is saved as `data/uploaded_custom.csv` and, until one is uploaded, the reference dataset is used. The loader returns the list of baskets, from which the dashboard shows the number of active transactions and unique products, and it offers every product in the cart selector.

5.4 Data Preprocessing

The preprocessing described in Chapter 3 is implemented in `load_transactions()` (reading, layout detection, filtering, title-casing and set-based de-duplication) and in the two download functions of `fetch_public_data.py` (conversion of the raw files to the standard two-column format, removal of cancelled invoices and non-product lines, and the 3,000-invoice limit for the UCI data).

5.5 Engine Structure and Functions

Table 5.2: Functions of the mining engine (main.py)

Function	Input	Output	Purpose
`load_transactions`	CSV file path	List of sorted item lists	Universal loader and cleaner.
`append_transaction`	File path, items, optional ID	Transaction ID	Appends a live POS basket to the CSV file.
`apriori`	Baskets, minimum support, telemetry flag	Dictionary itemset to support (and telemetry list)	Frequent-itemset mining with pruning.
`generate_rules`	Frequent itemsets, baskets, minimum confidence, minimum lift	List of rule dictionaries sorted by lift	Rule generation with five measures.
`recommend`	Cart items, rules	Ranked list of recommendations	Next-best-offer for a basket.
`print_rules`, `main`	Rules / none	Console output	Command-line mode.

5.6 Implementation of the Mining Modules

5.6.1 Apriori engine

`apriori()` first converts every basket to a frozenset and counts all items with a single `Counter` pass to obtain the frequent 1-itemsets. For each later level it merges every pair of frequent (k−1)-itemsets by set union and keeps the unions of size k as raw candidates. A candidate survives pruning only if every one of its (k−1)-subsets is a key of the dictionary of frequent itemsets. The survivors are counted with a subset test against every frozenset basket, and those reaching the minimum support are added to the dictionary. The loop ends when no candidate survives, and the per-level telemetry is returned together with the itemsets when requested. Merging by union, rather than by the classical shared-prefix join, is simple and complete, because every frequent k-itemset is the union of two of its frequent (k−1)-subsets, but it compares more pairs of itemsets than a prefix join would.

5.6.2 Rule generation

`generate_rules()` loops over the frequent itemsets with at least two items and over every non-empty proper subset as antecedent. The support of the antecedent and of the consequent are read from the dictionary, so rule generation involves no pass over the transactions (the baskets are accepted as an argument for interface consistency but never re-scanned). Confidence, lift, leverage and conviction are computed from these values, conviction is capped at 99.99, and a rule is kept if both the confidence and the lift thresholds are met. The result is sorted by lift and then confidence.

5.6.3 Recommender

The `recommend()` function normalises the cart items in the same way as the loader, finds every rule whose antecedent is a subset of the cart, and keeps for each new product the rule with the highest confidence and lift. It returns the product, confidence, lift, support and the antecedent that triggered it.

5.6.4 Live POS checkout

The Complete Checkout button calls `append_transaction()`, which writes the cart as new rows with an identifier of the form `LIVE_nnnn` (the number is the count of existing baskets plus one) to the active dataset file. The dashboard then clears its Streamlit cache and reruns, so the rules are re-mined on the enlarged dataset and the recommendations update immediately. The Simulate 15 Live Store Baskets button appends fifteen random baskets of two to four products in the same way, to stress-test the update loop. Because the baskets are written into the dataset file itself, they become part of the data for all later runs (see Section 6.10).

5.7 Front-End Dashboard

The dashboard (`app.py`) has a sidebar and a main area. The sidebar holds the dataset selector and three sliders for minimum support, confidence and lift. Mining is wrapped in a cached function keyed by the file path, the file modification time and the three thresholds, so that moving a slider re-mines but repeating the same settings is instant, and appending a transaction invalidates the cache. Above the tabs a row of five indicators shows the number of active transactions, unique products, frequent itemsets, association rules and the mining time together with the number of pruned candidates. The four tabs are:

- **Live POS Checkout and Recommender:** a multi-select cart, the checkout and simulation buttons, and a table of next-best-offer recommendations with confidence, lift, rule support and the triggering items, plus a one-line promotion suggestion for the best recommendation.

- **Association Rules Explorer:** the table of all rules with the five measures, a filter by product, and a button that exports the table to `association_rules.csv`.
- **Visual Analytics and Co-Occurrence:** a horizontal bar chart of the twelve most frequent products and a bubble chart of support against confidence in which bubble size and colour show lift.
- **Algorithm Telemetry (Apriori Pruning):** the table of candidates generated, evaluated, eliminated and frequent itemsets for each level.

5.8 Export of Results

Rules shown in the explorer can be downloaded as a CSV file containing the antecedent, consequent, support (%), confidence (%), lift, leverage and conviction. In command-line mode the first 25 rules are printed as a formatted table together with the dataset size and the mining time.

5.9 Verification and Testing

The submitted project does not include an automated test suite. To establish that the from-scratch engine is correct, three independent checks were carried out for this report and all passed:

1. **Exhaustive enumeration on the reference dataset.** All 63 non-empty subsets of the six products were counted directly, and the 13 itemsets reaching 20% support were compared with the engine output. The sets and their supports are identical.
2. **Direct recount on the supermarket dataset.** The support of each of the 122 frequent itemsets returned by the engine was recomputed by scanning all 9,835 baskets. Every value agrees.
3. **Completeness of the frequent pairs.** All item pairs in the supermarket baskets were counted independently. The 61 pairs reaching 2% support are exactly the 61 frequent 2-itemsets reported by the engine, so no frequent pair was lost by the pruning step.

In addition, the telemetry produced on the supermarket data (169, 1,711, 477 and 1 candidates, of which 0, 0, 409 and 1 were pruned) is the same as the empirical table published in the project README.

5.10 Implementation Pipeline

The end-to-end flow is as follows:

1. The user selects a benchmark dataset or uploads a CSV file in the dashboard; missing benchmark files are downloaded automatically.
2. `load_transactions()` detects the layout, cleans the rows and returns the list of baskets.
3. `apriori()` finds the frequent itemsets and the pruning telemetry for the chosen minimum support.
4. `generate_rules()` creates the rules and measures them with support, confidence, lift, leverage and conviction.
5. The dashboard displays the key indicators, the rules, the charts and the telemetry, and `recommend()` produces next-best-offers for the cart.
6. A completed checkout is appended to the dataset by `append_transaction()`, the cache is cleared and the pipeline reruns.
7. The user can export the rules as a CSV file.

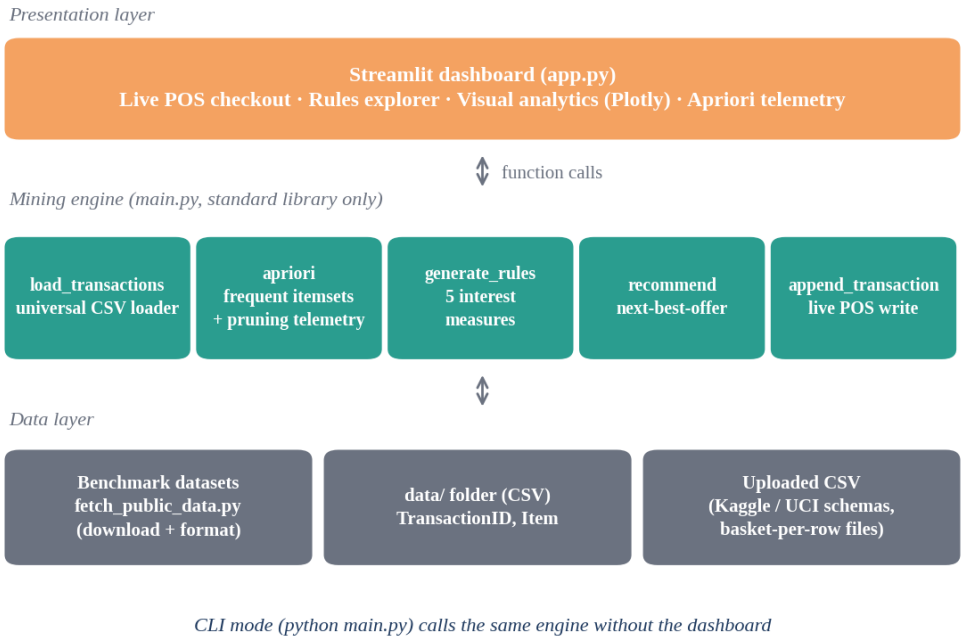


Figure 5.1: System Architecture of the Retail Market-Basket System

The architecture has three layers. The dashboard in the presentation layer calls the functions of the mining engine, and the engine reads from, and in live mode writes to, the CSV files of the data layer. The same engine can be used from the command line without the dashboard.

CHAPTER 6
RESULTS

6.1 Introduction

This chapter reports the output of the implemented engine. All figures were produced by running the project's own `load_transactions`, `apriori` and `generate_rules` functions on the original baskets of the three datasets (the added LIVE baskets removed, Section 2.1). The thresholds are those in Table 4.2. Run times are medians of three runs on the machine used to prepare this report and will differ on other hardware. Because the datasets are public benchmarks, the results show what the system discovers and how it behaves, not the behaviour of a specific store.

6.2 Data Analysis

In the supermarket data a few products dominate. Whole Milk appears in 25.6% of baskets, Other Vegetables in 19.4%, Rolls/Buns in 18.4% and Soda in 17.4% (Table 6.1, Figure 6.1). Only 59 of the 169 product categories reach the 2% support threshold, so about 35% of the categories take part in any frequent itemset. The product popularity is strongly skewed, which matters when reading confidence values: any rule whose consequent is Whole Milk starts with a 25.6% chance of being right by luck.

Table 6.1: Ten most frequent products in the supermarket dataset

Rank	Product	Baskets	Share of baskets (%)
1	Whole Milk	2,513	25.55
2	Other Vegetables	1,903	19.35
3	Rolls/Buns	1,809	18.39
4	Soda	1,715	17.44
5	Yogurt	1,372	13.95
6	Bottled Water	1,087	11.05
7	Root Vegetables	1,072	10.90
8	Tropical Fruit	1,032	10.49
9	Shopping Bags	969	9.85
10	Sausage	924	9.40

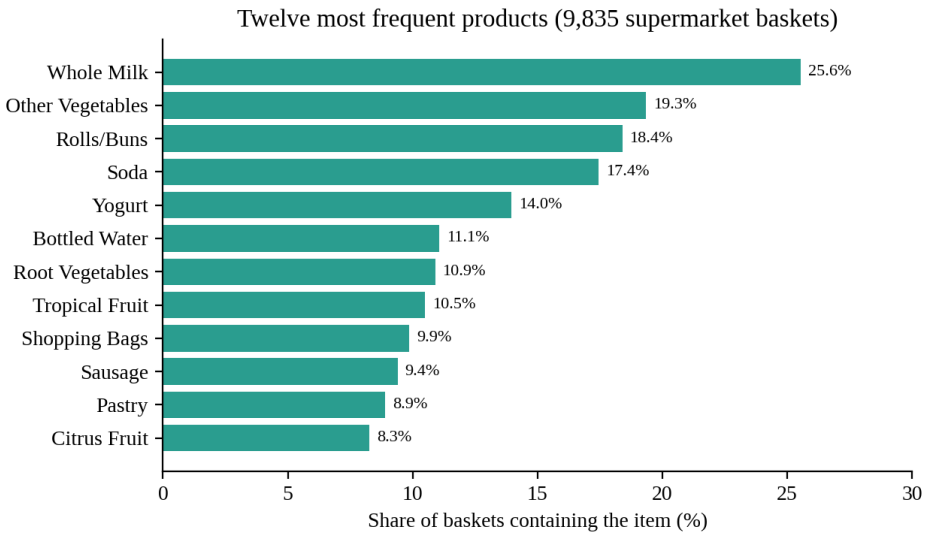


Figure 6.1: Most Frequent Products in the Supermarket Dataset

The UCI invoices are different in character. The most frequent product, White Hanging Heart T-Light Holder, appears in 15.2% of invoices, followed by Regency Cakestand 3 Tier (11.4%) and Heart Of Wicker Small (10.7%). With 2,955 distinct products and large baskets, the frequencies are spread over many more items, and only 374 products reach 2% support.

6.3 Results on the Reference Dataset

On the 15-basket reference dataset with minimum support 20%, confidence 50% and lift 1.0, the engine found 13 frequent itemsets (6 single items, 6 pairs and the triple Bread, Butter and Milk) and 13 rules in 0.2 ms (Table 6.2). The strongest rules are Coffee → Sugar and Sugar → Coffee, with confidence 100%, lift 5.0 and conviction at the 99.99 cap: whenever one of the two is bought, the other is always in the basket. The Bread, Butter and Milk rules have lifts only slightly above 1 because Milk and Bread are bought in a majority of all baskets, so little more than chance is explained.

Table 6.2: Association rules on the reference dataset (support 20%, confidence 50%, lift 1.0)

#	Antecedent	Consequent	Support (%)	Conf. (%)	Lift	Leverage	Conv.
1	Coffee	Sugar	20.00	100.0	5.00	0.1600	99.99
2	Sugar	Coffee	20.00	100.0	5.00	0.1600	99.99
3	Bread	Butter	40.00	66.7	1.25	0.0800	1.40
4	Butter	Bread	40.00	75.0	1.25	0.0800	1.60
5	Eggs	Bread	20.00	75.0	1.25	0.0400	1.60
6	Butter, Milk	Bread	26.67	66.7	1.11	0.0267	1.20
7	Bread, Milk	Butter	26.67	57.1	1.07	0.0178	1.09
8	Butter	Bread, Milk	26.67	50.0	1.07	0.0178	1.07
9	Bread	Milk	46.67	77.8	1.06	0.0267	1.20

#	Antecedent	Consequent	Support (%)	Conf. (%)	Lift	Leverage	Conv.
10	Milk	Bread	46.67	63.6	1.06	0.0267	1.10
11	Milk	Butter	40.00	54.5	1.02	0.0089	1.03
12	Butter	Milk	40.00	75.0	1.02	0.0089	1.07
13	Eggs	Milk	20.00	75.0	1.02	0.0044	1.07

6.4 Frequent Itemsets and Apriori Pruning

With minimum support 2%, the supermarket data yields 122 frequent itemsets: 59 single items, 61 pairs and 2 triples, and no frequent itemset of four items. The UCI data yields 516: 374 items, 140 pairs and 2 triples. Table 6.3 shows the telemetry for the three datasets and Figure 6.2 plots it for the two real ones.

Table 6.3: Apriori candidate generation and pruning telemetry

Dataset	Level k	Candidates generated	Evaluated after pruning	Pruned	Frequent itemsets
Reference	1	6	6	0	6
	2	15	15	0	6
	3	4	2	2	1
Supermarket POS	1	169	169	0	59
	2	1,711	1,711	0	61
	3	477	68	409	2
	4	1	0	1	0
UCI Online Retail	1	2,955	2,955	0	374
	2	69,751	69,751	0	140
	3	710	69	641	2
	4	1	0	1	0

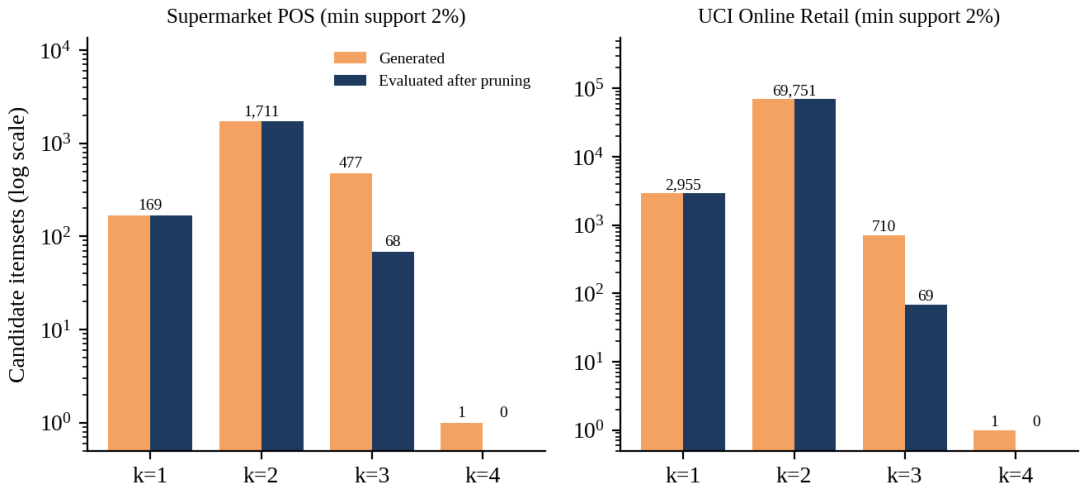


Figure 6.2: Candidate Itemsets Generated and Evaluated at Each Level

At level 2 nothing can be pruned, because every pair of frequent items is a candidate (59 items give 1,711 pairs, 374 give 69,751). From level 3 the Apriori property takes effect. In the supermarket data, 409 of the 477 candidate triples (85.7%) are discarded because at least one of their pairs is infrequent, so only 68 triples are counted against the baskets; the single candidate four-itemset is also pruned. In the UCI data 641 of 710 triples (90.3%) are pruned and 69 are counted. Over all levels, 410 of 2,358 supermarket candidates (17.4%) and 642 of 73,417 UCI candidates (0.9%) never needed a transaction scan. The overall percentage is small because the level-2 candidates dominate the total, but the pruning is what stops the search from continuing to larger and larger itemsets.

6.5 Association Rules on the Supermarket Dataset

With minimum support 2%, confidence 30% and lift 1.2, the engine produced 37 rules (32 with one item on each side and 5 with two antecedent items), in 0.68 seconds. Support ranges from 2.0% to 7.5%, confidence from 30.8% to 51.3%, and lift from 1.21 to 2.84. Table 6.4 lists the ten strongest rules by lift, and Figures 6.3 and 6.4 show all rules and the top ten.

Table 6.4: Ten strongest rules by lift, supermarket dataset

#	Antecedent	Consequent	Support (%)	Conf. (%)	Lift	Leverage	Conv.
1	Other Vegetables, Whole Milk	Root Vegetables	2.32	31.0	2.84	0.0150	1.29
2	Root Vegetables, Whole Milk	Other Vegetables	2.32	47.4	2.45	0.0137	1.53
3	Root Vegetables	Other Vegetables	4.74	43.5	2.25	0.0263	1.43
4	Whipped/Sour Cream	Other Vegetables	2.89	40.3	2.08	0.0150	1.35
5	Whole Milk, Yogurt	Other Vegetables	2.23	39.7	2.05	0.0114	1.34
6	Other Vegetables, Yogurt	Whole Milk	2.23	51.3	2.01	0.0112	1.53

#	Antecedent	Consequent	Support (%)	Conf. (%)	Lift	Leverage	Conv.
7	Butter	Whole Milk	2.76	49.7	1.95	0.0134	1.48
8	Pork	Other Vegetables	2.17	37.6	1.94	0.0105	1.29
9	Curd	Whole Milk	2.61	49.0	1.92	0.0125	1.46
10	Other Vegetables, Root Vegetables	Whole Milk	2.32	48.9	1.91	0.0111	1.46

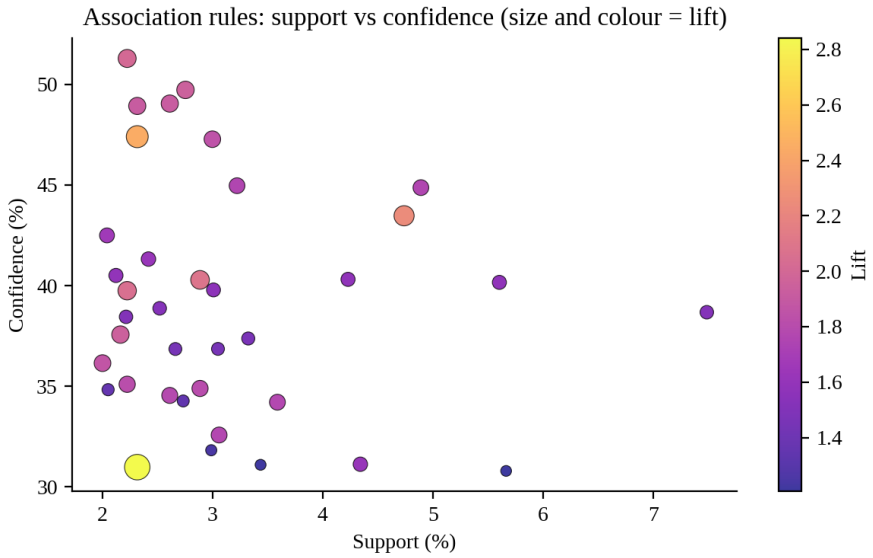


Figure 6.3: Support versus Confidence of the Supermarket Rules

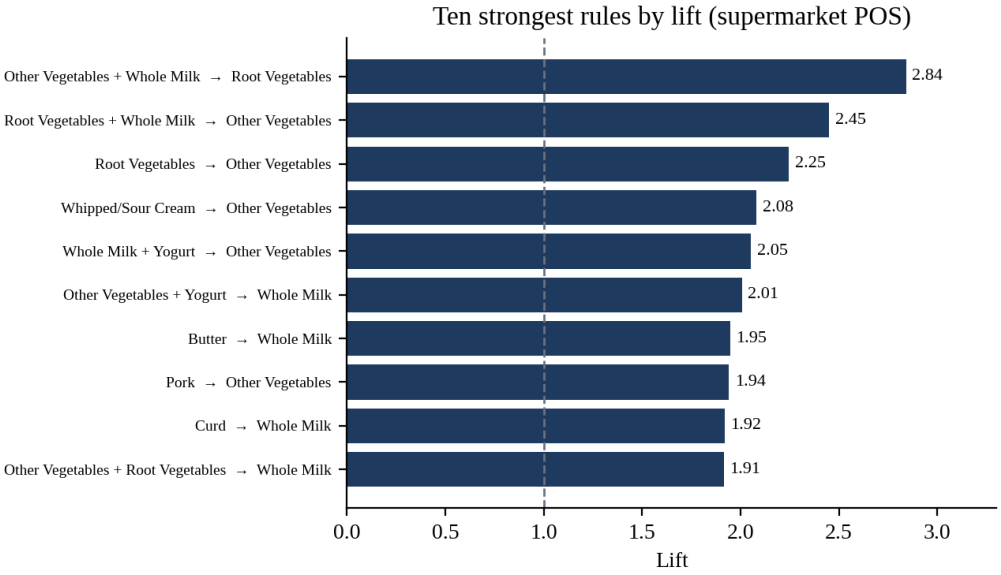


Figure 6.4: Ten Strongest Supermarket Rules by Lift

Several patterns can be read from the rules:

- **Whole Milk and Other Vegetables are the hubs.** Whole Milk appears in 27 of the 37 rules (as the consequent in 24) and Other Vegetables in 15 (as the consequent in 11). These are the two most frequent categories, so they co-occur with many other products.
- **The strongest associations involve fresh produce.** The three highest-lift rules all link Root Vegetables, Other Vegetables and Whole Milk: for example baskets with Other Vegetables and Whole Milk are 2.84 times as likely as average to contain Root Vegetables, and baskets with Root Vegetables and Whole Milk are 2.45 times as likely to contain Other Vegetables.
- **Dairy and eggs predict whole milk.** Butter, Curd, Whipped/Sour Cream, Yogurt and Domestic Eggs each imply Whole Milk with confidence between 40% and 50% and lift between 1.57 and 1.95.
- **Rules are moderate in strength.** The highest confidence is 51.3% (Other Vegetables and Yogurt imply Whole Milk), so even the best rule is correct for about half of the matching baskets. The leverage values are small (at most 0.026) because the baskets are small and sparse.

6.6 Association Rules on the UCI Online Retail Dataset

With minimum support 2%, confidence 40% and lift 1.5, the UCI data produced 132 rules (126 one-to-one and 6 with two antecedent items) in 9.1 seconds. The rules are much stronger than in the supermarket data: confidence ranges from 40.1% to 83.1% and lift from 2.7 to 27.4. Table 6.5 and Figure 6.5 show the ten strongest. Each pair appears twice, once in each direction, with the same lift and support.

Table 6.5: Ten strongest rules by lift, UCI Online Retail dataset

#	Antecedent	Consequent	Support (%)	Conf. (%)	Lift	Leverage	Conv.
1	Jam Jar With Green Lid	Jam Jar With Pink Lid	2.13	83.1	27.40	0.0206	5.74
2	Jam Jar With Pink Lid	Jam Jar With Green Lid	2.13	70.3	27.40	0.0206	3.28
3	Poppy'S Playhouse Bedroom	Poppy'S Playhouse Kitchen	2.23	74.4	24.01	0.0214	3.79
4	Poppy'S Playhouse Kitchen	Poppy'S Playhouse Bedroom	2.23	72.0	24.01	0.0214	3.47
5	Blue 3 Piece Polkadot Cutlery Set	Pink 3 Piece Polkadot Cutlery Set	2.13	71.9	21.57	0.0203	3.44
6	Pink 3 Piece Polkadot Cutlery Set	Blue 3 Piece Polkadot Cutlery Set	2.13	64.0	21.57	0.0203	2.70
7	Feltcraft Cushion Butterfly	Feltcraft Cushion Rabbit	2.03	61.0	18.30	0.0192	2.48
8	Feltcraft Cushion Rabbit	Feltcraft Cushion Butterfly	2.03	61.0	18.30	0.0192	2.48

#	Antecedent	Consequent	Support (%)	Conf. (%)	Lift	Leverage	Conv.
9	Small Heart Measuring Spoons	Large Heart Measuring Spoons	2.50	57.7	15.45	0.0234	2.28
10	Large Heart Measuring Spoons	Small Heart Measuring Spoons	2.50	67.0	15.45	0.0234	2.90

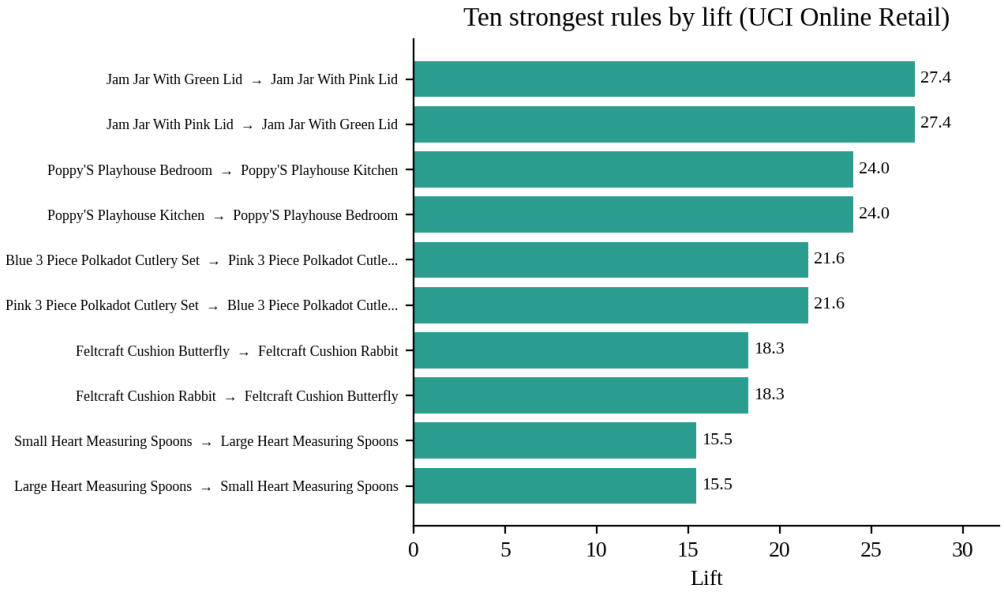


Figure 6.5: Ten Strongest UCI Rules by Lift

The strongest UCI rules link variants of the same product family: Jam Jar With Green Lid and With Pink Lid (lift 27.4, confidence up to 83.1%), the Poppy's Playhouse Bedroom and Kitchen toys (24.0), the blue and pink Polkadot cutlery sets (21.6), the Feltcraft cushions (18.3), and the small and large measuring spoons (15.5). Customers who buy one colour or size often buy the other, which is plausible for a gift retailer whose customers buy sets. Rules between different product families also exist, for example Candleholder Pink Hanging Heart implies White Hanging Heart T-Light Holder with confidence 72.5% and lift 4.8. The very high lifts are partly a consequence of the sparse catalogue: these are niche products (about 2% support), so even a moderate number of joint purchases is far above what chance predicts.

6.7 Recommendation Results

Table 6.6 shows the recommendations produced for four sample carts on the supermarket rules. For each cart the products are ranked by confidence, and the triggering antecedent is shown.

Table 6.6: Next-best-offer recommendations for sample carts (supermarket dataset)

Cart	Recommended product	Confidence (%)	Lift	Triggered by
Yogurt	Whole Milk	40.2	1.57	Yogurt

Cart	Recommended product	Confidence (%)	Lift	Triggered by
	Other Vegetables	31.1	1.61	Yogurt
Root Vegetables, Whole Milk	Other Vegetables	47.4	2.45	Root Vegetables, Whole Milk
Butter	Whole Milk	49.7	1.95	Butter
	Other Vegetables	36.1	1.87	Butter
Sausage	Rolls/Buns	32.6	1.77	Sausage
	Whole Milk	31.8	1.25	Sausage

A cart containing Root Vegetables and Whole Milk receives a single recommendation, Other Vegetables, at 47.4% confidence, because Whole Milk is already in the cart and so is not recommended. The rule with two antecedent items takes priority over the one-item rule Root Vegetables to Other Vegetables (43.4%) since its confidence is higher. A cart with Butter gets Whole Milk (49.7%) and Other Vegetables (36.1%).

6.8 Sensitivity to the Thresholds

Table 6.7 and Figure 6.6 show how the minimum support changes the result on the supermarket data (confidence 30%, lift 1.2). Halving support from 2% to 1% increases the frequent itemsets from 122 to 333, the rules from 37 to 125 and the run time from 0.72 to 1.76 seconds, while raising it to 5% leaves only 31 itemsets and 3 rules. The proportion of candidates removed by pruning also grows as support decreases, from 0.5% at 5% support to 35.1% at 1%.

Table 6.7: Effect of minimum support (supermarket dataset, confidence 30%, lift 1.2)

Minimum support	Frequent itemsets	Rules	Candidates generated	Pruned	Time (ms)
1%	333	125	7,058	2,479	1,755
1.5%	180	59	3,812	857	1,116
2%	122	37	2,358	410	721
3%	63	14	1,185	61	409
5%	31	3	550	3	222

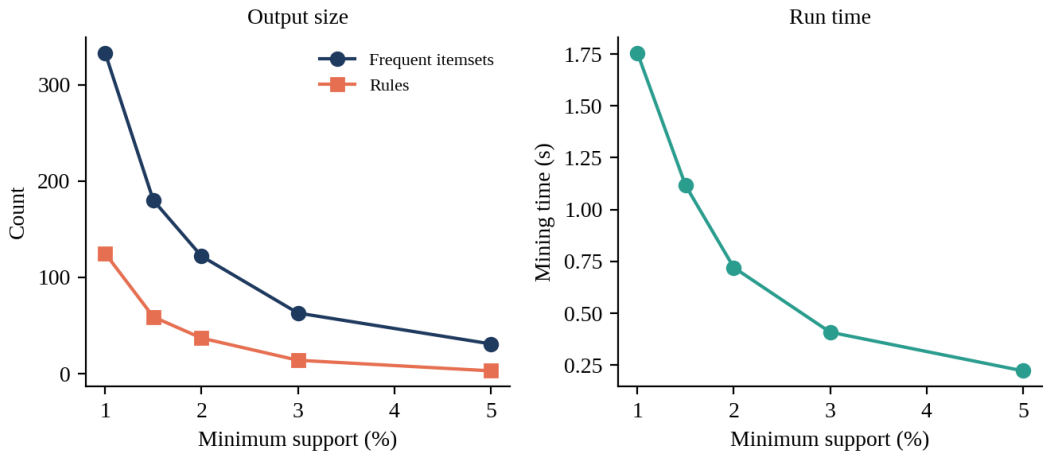


Figure 6.6: Effect of Minimum Support on Output Size and Run Time

Table 6.8 shows the effect of confidence and lift at 2% support. Raising the confidence threshold from 30% to 50% reduces the rules from 37 to 1, and no rule reaches 60%, which confirms that supermarket associations are moderate. The lift threshold has no effect between 1.0 and 1.2 because every rule with 30% confidence already has lift of at least 1.2, but a threshold of 2.0 keeps only 6 rules and 2.5 keeps only the single strongest.

Table 6.8: Effect of minimum confidence and minimum lift (supermarket dataset, support 2%)

Minimum confidence (lift 1.2)	Rules	Minimum lift (confidence 30%)	Rules
20%	64	1.0	37
30%	37	1.2	37
40%	15	1.5	29
50%	1	2.0	6
60%	0	2.5	1

6.9 Performance and Verification

Table 6.9: Mining time and workload

Dataset	Baskets	Level-2 candidates	Subset tests at level 2	Mining time (frequent itemsets and rules)
Reference	15	15	225	0.2 ms
Supermarket POS	9,835	1,711	16.8 million	678 ms
UCI Online Retail	3,000	69,751	209.3 million	9.1 s

The UCI run takes about 13 times as long as the supermarket run, in line with the 12-fold increase in subset tests at level 2. Performance is therefore driven by the number of frequent items rather than by the number of baskets, which is why the support threshold is the most

important parameter. The verification checks of Section 5.9 all passed: the 13 reference itemsets match exhaustive enumeration, the 122 supermarket supports match a direct recount, and the 61 frequent pairs match an independent pair count.

6.10 Effect of Live Transactions

The shipped supermarket file contains 28 LIVE baskets (85 rows) left from dashboard testing, and the reference file contains one. With them included, the supermarket dataset has 9,863 baskets and mining with the same thresholds gives 121 frequent itemsets and 36 rules, compared with 122 and 37 on the original 9,835 baskets: one frequent itemset and one rule drop out because they fall just below their thresholds. The change is small here because 28 baskets are only 0.3% of the data, but it shows that live and simulated baskets (which are random in the Simulate button) become permanent parts of the dataset and can dilute or alter real patterns, particularly in small datasets. For the 15-basket reference dataset a single added basket changes all supports noticeably. This is why the results in this report exclude them.

6.11 Summary of Results

Table 6.10: Summary of mining results

Dataset	Settings (support / confidence / lift)	Frequent itemsets	Rules	Key result
Reference (15 baskets)	20% / 50% / 1.0	13	13	Coffee and Sugar always bought together (lift 5.0)
Supermarket POS (9,835 baskets)	2% / 30% / 1.2	122	37	Top rule lift 2.84; Whole Milk in 27 rules; 409 of 477 triples pruned
UCI Online Retail (3,000 invoices)	2% / 40% / 1.5	516	132	Lift up to 27.4, mainly product variants; 641 of 710 triples pruned

CHAPTER 7
COMPARISON AND DISCUSSION

7.1 Comparison of the Datasets

The same engine behaves very differently on the two real datasets (Table 7.1), and the differences explain much about how association rules should be interpreted.

Table 7.1: Comparison of the results on the three datasets

Aspect	Reference	Supermarket POS	UCI Online Retail
Baskets / distinct items	15 / 6	9,835 / 169	3,000 / 2,955
Mean basket size	2.5	4.4	27.3
Frequent itemsets	13	122	516
Rules	13	37	132
Lift range of rules	1.02 to 5.0	1.21 to 2.84	2.7 to 27.4
Confidence range of rules	50% to 100%	30.8% to 51.3%	40.1% to 83.1%
Nature of the rules	Planted pair plus staples	Staple categories around Whole Milk and vegetables	Colour and size variants of the same product
Mining time	0.2 ms	678 ms	9.1 s

The supermarket rules are broad and moderate: they describe staple products that many customers buy, which gives high support but modest lift. The UCI rules are narrow and strong: they describe niche products with a very specific complementary relationship. A retailer would use them differently, the first for category-level planning such as placing dairy near fresh produce, and the second for product-level bundles and "complete the set" prompts.

7.2 Choosing the Thresholds

Minimum support is a trade-off between coverage and cost. At 5% only three supermarket rules survive, at 1% there are 125 and the run time almost doubles relative to 2% (Table 6.7). Confidence should be read together with lift: Whole Milk is in 25.6% of baskets, so a rule that predicts it with 40% confidence (lift 1.57) is a real but modest association, while a rule predicting a rare product with the same confidence would be far more striking. Leverage and conviction give additional views, but on sparse data leverage values are small in absolute terms, and conviction depends strongly on the support of the consequent. In practice an analyst would start with a low support to see the shape of the data and then raise the lift threshold to keep the most informative rules.

7.3 Apriori Compared with Alternative Algorithms

Table 7.2: Apriori compared with other frequent-itemset algorithms

Aspect	Apriori (this project)	FP-Growth	Eclat
Search strategy	Level-wise candidate generation and test	Pattern growth on a compressed prefix tree, no candidates	Depth-first search with transaction-ID list intersection
Passes over the data	Classically one pass per level; this implementation tests every candidate against every basket	Two passes to build the tree	One pass to build ID lists
Memory	Low: only the current candidates and itemsets	Needs the whole tree in memory	Needs the ID lists in memory
Strength	Simple and transparent; pruning is easy to measure and explain	Usually much faster on large dense data	Fast on sparse data, simple intersections
Weakness	Slow when many frequent items create many candidate pairs	More complex to implement and explain	ID lists can become large on dense data

Apriori was chosen because the aim of the project is to show and measure the algorithm itself, including the pruning that gives it its name. The telemetry in Table 6.3 does exactly that. The run times of 0.7 seconds for the supermarket data and 9 seconds for the UCI data show that the pure-Python implementation is adequate for datasets of this size but that FP-Growth or ID-list methods would be preferable for much larger data.

7.4 Advantages

- **Transparent, from-scratch engine:** the pruning step, the support counting and the rule measures are all visible in about 130 lines of code, and no mining library is needed.
- **Measured pruning:** the per-level telemetry shows precisely how many candidates the Apriori property removes.
- **Constant-time rule generation:** antecedent and consequent supports come from the dictionary of frequent itemsets and no further scan is required.
- **Five complementary measures:** support, confidence, lift, leverage and conviction allow rules to be judged from different angles.
- **Flexible input:** several Kaggle and UCI layouts and headerless basket files load without any manual mapping.
- **Interactive and real-time:** thresholds can be changed with sliders and a live checkout updates the rules and recommendations immediately.
- **Real public data:** results can be reproduced by anyone who downloads the same datasets.

7.5 Limitations

- **Presence only:** quantities, prices, margins and time are ignored, so a rule between two cheap products and one between two high-margin products are treated alike.
- **Item granularity:** the supermarket items are broad product categories, so the rules are category-level and cannot say which brand of milk or yogurt to promote. In the UCI data the strongest rules are between variants of the same product, which are often obvious.
- **Rule strength:** on the supermarket data no rule has a confidence above 51.3% or a lift above 2.84, so the rules describe tendencies rather than reliable predictions. They show association, not causation, and any promotion based on them should be tested.
- **Few multi-item rules:** at 2% support only two frequent triples exist in each real dataset, so almost all rules have a single item on each side.
- **Speed:** support counting tests every candidate against every basket without hash trees or ID lists, and merging pairs of frequent itemsets examines more pairs than a prefix join. This is acceptable for the datasets used (up to 9 seconds) but would not scale to millions of baskets.
- **Live data written into the dataset file:** `append_transaction()` writes live and simulated baskets into the active CSV, so they permanently alter the source data (28 such baskets are present in the shipped supermarket file). Random simulated baskets add noise. A separate file or database table for live data would be safer.
- **Loader assumptions:** transaction identifiers starting with the letter C are treated as cancelled invoices, which would also remove any other identifier beginning with C. In the Kaggle Groceries layout the identifier is a customer number, not a single receipt, so mining by it merges all visits of a customer into one basket. Title-casing leaves small artefacts such as Poppy'S.
- **Sampling of the UCI data:** only the first 3,000 valid invoices are used and these are not a random sample, and many of them are wholesale-sized baskets.
- **No automated test suite:** correctness was verified with the checks of Section 5.9 but these are not part of the submitted code.
- **Public data only:** the results have not been validated on the transactions of a real retailer or against sales outcomes.

CHAPTER 8
CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

This project developed a Retail Market-Basket and POS Analytics System that mines association rules from retail transactions with an Apriori engine written from scratch. The engine finds frequent itemsets with downward-closure pruning, records the effect of the pruning at every level, generates rules measured by support, confidence, lift, leverage and conviction using constant-time lookups, and converts the rules into real-time product recommendations. A universal loader handles the main public transaction layouts, and a Streamlit dashboard provides a live checkout, a rules explorer, visual analytics and algorithm telemetry.

On the 9,835-basket supermarket dataset the system found 122 frequent itemsets and 37 rules at 2% support, 30% confidence and lift of 1.2 in about 0.7 seconds. The rules revolve around Whole Milk and vegetables, with a maximum lift of 2.84 and a maximum confidence of 51.3%, and the Apriori property eliminated 85.7% of the candidate triples without counting them. On 3,000 UCI Online Retail invoices it found 516 frequent itemsets and 132 rules with lifts up to 27.4, mostly between variants of the same product. The engine was verified against exhaustive enumeration and direct recounts, and every number in this report comes from running the submitted code. The project therefore shows how a standard data mining algorithm can turn raw receipts into concrete, measurable cross-selling knowledge, and how the choice of thresholds shapes what is found.

8.2 Future Scope

- Add FP-Growth and Eclat implementations and compare their run times and memory use with Apriori on larger datasets.
- Speed up support counting with transaction-ID bitsets or hash trees, and a prefix-based candidate join.
- Store live transactions in a separate file or database table so that the benchmark data is never modified, and add authentication and multi-store support.
- Add an automated pytest suite built from the verification checks of Section 5.9.
- Include quantities, prices and margins through high-utility itemset mining, so rules can be ranked by profit and not only by frequency.

REFERENCES

[1] R. Agrawal, T. Imielinski and A. Swami, "Mining association rules between sets of items in large databases," in Proc. ACM SIGMOD International Conference on Management of Data, 1993, pp. 207-216.

[2] R. Agrawal and R. Srikant, "Fast algorithms for mining association rules," in Proc. 20th International Conference on Very Large Data Bases (VLDB), 1994, pp. 487-499.

[3] J. Han, M. Kamber and J. Pei, Data Mining: Concepts and Techniques, 3rd ed. Morgan Kaufmann, 2011.

[4] S. Brin, R. Motwani, J. D. Ullman and S. Tsur, "Dynamic itemset counting and implication rules for market basket data," in Proc. ACM SIGMOD International Conference on Management of Data, 1997, pp. 255-264.

[5] G. Piatetsky-Shapiro, "Discovery, analysis, and presentation of strong rules," in Knowledge Discovery in Databases, G. Piatetsky-Shapiro and W. J. Frawley, Eds. AAAI/MIT Press, 1991, pp. 229-248.

[6] J. Han, J. Pei and Y. Yin, "Mining frequent patterns without candidate generation," in Proc. ACM SIGMOD International Conference on Management of Data, 2000, pp. 1-12.

[7] M. J. Zaki, "Scalable algorithms for association mining," IEEE Transactions on Knowledge and Data Engineering, vol. 12, no. 3, pp. 372-390, 2000.

[8] M. Hahsler, K. Hornik and T. Reutterer, "Implications of probabilistic data modeling for mining association rules," in From Data and Information Analysis to Knowledge Engineering. Springer, 2006, pp. 598-605.

[9] D. Chen, S. L. Sain and K. Guo, "Data mining for the online retail industry: A case study of RFM model-based customer segmentation using data mining," Journal of Database Marketing & Customer Strategy Management, vol. 19, no. 3, pp. 197-208, 2012.

[10] D. Dua and C. Graff, UCI Machine Learning Repository, University of California, Irvine, School of Information and Computer Sciences. [Online]. Available: https://archive.ics.uci.edu

[11] B. Lantz, Machine Learning with R, Packt Publishing. Groceries dataset repository. [Online]. Available: https://github.com/stedy/Machine-Learning-with-R-datasets

[12] W. McKinney, "Data structures for statistical computing in Python," in Proc. 9th Python in Science Conference, 2010, pp. 56-61.

[13] Streamlit documentation. [Online]. Available: https://docs.streamlit.io

APPENDIX: KEY SOURCE CODE

The core of the mining engine (`main.py`) and the cached mining call of the dashboard (`app.py`) are reproduced below exactly as submitted. The complete project (engine, dashboard, dataset downloader, documentation and data) is submitted with this report in the project archive.

A.1 Universal transaction loader (load_transactions)

```
def load_transactions(file_path):
    """
    Universal transaction loader supporting standard CSVs plus raw Kaggle & UCI datasets:
    - Standard: (TransactionID, Item)
    - UCI Online Retail: (InvoiceNo, Description)
    - Kaggle Bread Basket: (Transaction, Item)
    - Kaggle Groceries: (Member_number, itemDescription)
    - Unheadered Basket-per-row CSVs
    """
    baskets = {}
    with open(file_path, "r", newline="", encoding="utf-8-sig", errors="ignore") as file:
        sample = file.read(2048)
        file.seek(0)
        reader = csv.DictReader(file)
        headers = {h.strip(): h for h in (reader.fieldnames or []) if h}


        # Auto-detect Transaction ID column and Item column across popular Kaggle/UCI datasets
        id_candidates = ["TransactionID", "InvoiceNo", "Transaction", "Member_number", "Order_ID",
"BillNo"]
        item_candidates = ["Item", "Description", "itemDescription", "Product", "Itemname"]


        id_col = next((headers[c] for c in id_candidates if c in headers), None)
        item_col = next((headers[c] for c in item_candidates if c in headers), None)


        if id_col and item_col:
            for row in reader:
                tid = str(row.get(id_col, "")).strip()
                item = str(row.get(item_col, "")).strip().title()
                # Filter out cancelled invoices or "None" items in Kaggle BreadBasket
                if tid and not tid.startswith("C") and item and item.lower() not in {"none",
"nan", "adjustment"}:
                    baskets.setdefault(tid, set()).add(item)
        else:
            # Fallback: Basket-per-row format (like raw groceries.csv)
            file.seek(0)
            raw_reader = csv.reader(file)
            for idx, row in enumerate(raw_reader, start=1):
                clean_items = {cell.strip().title() for cell in row if cell.strip()}
                if clean_items:
                    baskets[f"T{idx:04d}"] = clean_items


    return [sorted(items) for items in baskets.values() if items]
```

A.2 Live POS write (append_transaction)

```
def append_transaction(file_path, items, transaction_id=None):
    """Append a new real-time POS transaction to the CSV dataset."""
    clean_items = sorted({item.strip().title() for item in items if item.strip()})
    if not clean_items:
        return None


    if not transaction_id:
        existing = load_transactions(file_path) if Path(file_path).exists() else []
        transaction_id = f"LIVE_{len(existing) + 1:04d}"


    with open(file_path, "a", newline="", encoding="utf-8") as file:
        writer = csv.writer(file)
        for item in clean_items:
            writer.writerow([transaction_id, item])
```

```
    return transaction_id
```

A.3 Apriori frequent-itemset mining (apriori)

```
def apriori(transactions, min_support, return_telemetry=False):
    """
    Find frequent itemsets using the Apriori principle.
    Optimized using frozensets for fast subset checking and tracks pruning telemetry.
    """
    if not transactions:
        return ({}, []) if return_telemetry else {}


    num_txns = len(transactions)
    fast_baskets = [frozenset(b) for b in transactions]


    # Level 1: Single items counted in a single pass O(N)
    item_counts = Counter(item for basket in transactions for item in basket)
    frequent = {}
    current_level = []


    for item, count in sorted(item_counts.items()):
        support = count / num_txns
        if support >= min_support:
            itemset = (item,)
            frequent[itemset] = support
            current_level.append(itemset)


    telemetry = [{
        "level": 1,
        "candidates_generated": len(item_counts),
        "candidates_after_pruning": len(item_counts),
        "pruned_count": 0,
        "frequent_itemsets": len(current_level),
    }]


    # Levels k = 2, 3, ...
    while current_level:
        k = len(current_level[0]) + 1
        raw_candidates = set()
        pruned_candidates = set()


        for i in range(len(current_level)):
            for j in range(i + 1, len(current_level)):
                merged = tuple(sorted(set(current_level[i]) | set(current_level[j])))
                if len(merged) == k:
                    raw_candidates.add(merged)
                    # Apriori pruning: every (k-1)-subset MUST be frequent
                    subsets = combinations(merged, k - 1)
                    if all(tuple(sorted(sub)) in frequent for sub in subsets):
                        pruned_candidates.add(merged)


        if not pruned_candidates:
            if raw_candidates:
                telemetry.append({
                    "level": k,
                    "candidates_generated": len(raw_candidates),
                    "candidates_after_pruning": 0,
                    "pruned_count": len(raw_candidates),
                    "frequent_itemsets": 0,
                })
            break


        supported = []
        for candidate in sorted(pruned_candidates):
            cand_set = frozenset(candidate)
            matches = sum(1 for basket in fast_baskets if cand_set.issubset(basket))
            support = matches / num_txns
            if support >= min_support:
                frequent[candidate] = support
                supported.append(candidate)


        telemetry.append({
            "level": k,
            "candidates_generated": len(raw_candidates),
            "candidates_after_pruning": len(pruned_candidates),
            "pruned_count": len(raw_candidates) - len(pruned_candidates),
            "frequent_itemsets": len(supported),
```

```
        })

        current_level = supported

    if return_telemetry:
        return frequent, telemetry
    return frequent
```

A.4 Association rule generation (generate_rules)

```
def generate_rules(frequent_itemsets, transactions, min_confidence, min_lift):
    """
    Generate association rules with Support, Confidence, Lift, Leverage, and Conviction.
    Uses O(1) dictionary lookups via Apriori downward closure property.
    """
    rules = []
    for itemset, itemset_support in frequent_itemsets.items():
        if len(itemset) < 2:
            continue

        items = set(itemset)
        for size in range(1, len(itemset)):
            for antecedent_tuple in combinations(itemset, size):
                antecedent = tuple(sorted(antecedent_tuple))
                consequent = tuple(sorted(items - set(antecedent_tuple)))

                # O(1) Hash-Map Lookup (since all subsets of a frequent itemset are frequent)
                antecedent_support = frequent_itemsets.get(antecedent, 0.0)
                consequent_support = frequent_itemsets.get(consequent, 0.0)

                if antecedent_support == 0 or consequent_support == 0:
                    continue

                confidence = itemset_support / antecedent_support
                lift = confidence / consequent_support
                # Extra Data Mining Metrics for 3rd-Year AIDA Evaluation:
                # Leverage(X -> Y) = P(X and Y) - P(X)P(Y)
                leverage = itemset_support - (antecedent_support * consequent_support)
                # Conviction(X -> Y) = (1 - support(Y)) / (1 - confidence(X -> Y))
                conviction = (
                    (1.0 - consequent_support) / (1.0 - confidence)
                    if confidence < 1.0
                    else 99.99
                )

                if confidence >= min_confidence and lift >= min_lift:
                    rules.append({
                        "antecedent": antecedent,
                        "consequent": consequent,
                        "support": itemset_support,
                        "confidence": confidence,
                        "lift": lift,
                        "leverage": leverage,
                        "conviction": round(min(conviction, 99.99), 2),
                    })

    return sorted(rules, key=lambda r: (r["lift"], r["confidence"]), reverse=True)
```

A.5 Next-best-offer recommender (recommend)

```
def recommend(selected_items, rules):
    """Recommend products in real-time based on active cart items."""
    selected = {item.strip().title() for item in selected_items if item.strip()}
    recommendations = {}

    for rule in rules:
        if set(rule["antecedent"]).issubset(selected):
            for item in rule["consequent"]:
                if item not in selected:
                    current = recommendations.get(item)
                    if not current or (rule["confidence"], rule["lift"]) > (current["confidence"],
current["lift"]):
                        recommendations[item] = {
                            "item": item,
```

```
                            "confidence": rule["confidence"],
                            "lift": rule["lift"],
                            "support": rule["support"],
                            "based_on": ", ".join(rule["antecedent"]),
                        }

    return sorted(
        recommendations.values(),
        key=lambda r: (r["confidence"], r["lift"]),
        reverse=True,
    )
```

A.6 Cached mining call in the dashboard (app.py)

```
@st.cache_data(show_spinner="Mining frequent itemsets & rules...")
def run_cached_mining(file_path_str, file_mtime, min_sup, min_conf, min_lft):
    txns = load_transactions(file_path_str)
    freq, telem = apriori(txns, min_sup, return_telemetry=True)
    rls = generate_rules(freq, txns, min_conf, min_lft)
    return txns, freq, telem, rls
```